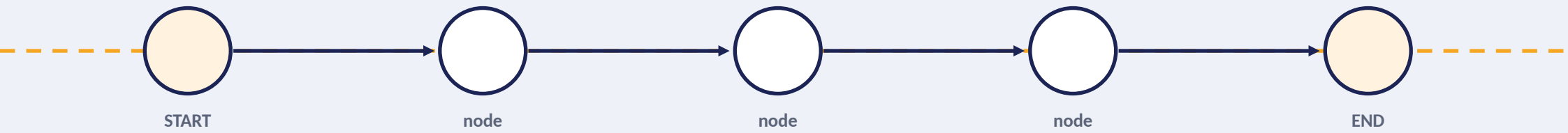
LANGGRAPH · CHAPTER 2 · FLASHCARDS

Your First Graph

State, nodes and edges

7 code cards on the three primitives and one complete run



The state schema

A TypedDict lists every field the graph carries from node to node

- The schema is the contract: nodes read these keys and return updates to them.
- Numbers, lists and strings all survive the trip; anything a node returns must be a key here.

PYTHON

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END


class SimpleState(TypedDict):
    input: str      # the text coming in
    output: str     # the text going out
    step: int       # a counter, to show numbers survive too
```

State in, partial update out

A node is a plain function: read the state, return only the keys you changed

- LangGraph merges the returned dict into the state; keys you leave out keep their values.
- Treat the incoming state as read-only: return a new dict instead of mutating it.

PYTHON

```python
def process(state: SimpleState) -> dict:
    output = state["input"].upper()
    step = state["step"] + 1
    # Only the changed keys: "input" stays as it was
    return {"output": output, "step": step}
```

Wire the graph, then compile it

add_node registers a name, add_edge wires names, compile() makes it runnable

- Edges reference the node's string name, not the function object.
- graph.set_entry_point("x") is the older spelling of add_edge(START, "x"); both still work.

PYTHON

```python
graph = StateGraph(SimpleState)      # 1. a builder bound to our schema
graph.add_node("process", process)   # 2. register the function by name
graph.add_edge(START, "process")     # 3. wire START -> process
graph.add_edge("process", END)       # 4. wire process -> END

app = graph.compile()                # blueprint -> runnable app
```

One run, one final state

invoke() takes the initial state and returns the merged final state

- The framework follows edges, merges each update and stops at END.
- You own the schema, the node logic and returning correct partial updates.

PYTHON

```python
initial_state = {"input": "hello", "output": "", "step": 0}
result = app.invoke(initial_state)
print(result)
# {'input': 'hello', 'output': 'HELLO', 'step': 1}

# Trace it by hand:
#   START -> process: "hello".upper() -> "HELLO", step 0 + 1 -> 1
#   process returns {"output": "HELLO", "step": 1}
#   LangGraph merges it, follows process -> END, returns the state
```

Two nodes, one baton

One node writes a field, the next node reads it: that is the whole handoff

- scale writes scaled; shift reads scaled and writes fahrenheit.
- The edge scale -> shift is the only thing that orders them.

PYTHON

```python
class TempState(TypedDict):
    celsius: float       # the input
    scaled: float        # written by scale, read by shift
    fahrenheit: float    # the result

def scale(state: TempState) -> dict:
    return {"scaled": state["celsius"] * 9 / 5}

def shift(state: TempState) -> dict:
    return {"fahrenheit": state["scaled"] + 32}

graph = StateGraph(TempState)
graph.add_node("scale", scale)
graph.add_node("shift", shift)
graph.add_edge(START, "scale")
graph.add_edge("scale", "shift")     # the handoff
graph.add_edge("shift", END)
print(graph.compile().invoke({"celsius": 25}))
# {'celsius': 25, 'scaled': 45.0, 'fahrenheit': 77.0}
```

Draw the graph you built

get_graph() gives a Mermaid definition and a PNG of the compiled wiring

- Draw it before you debug it: a missing edge is obvious in the picture.
- plot_graph lives in the repo's util.py and is reused in every chapter.

PYTHON

```python
def plot_graph(app, output_path="graph.png"):
    print(app.get_graph().draw_mermaid())          # text definition
    png_bytes = app.get_graph().draw_mermaid_png() # rendered image
    with open(output_path, "wb") as f:
        f.write(png_bytes)

plot_graph(app)   # START -> process -> END
```

Three quiet mistakes

A typo in a returned key or a missing END edge raises no error at all

- A key not in the schema is dropped silently; a wrong node name fails at compile().
- A node with no outgoing edge still compiles: the run simply stops after it.

PYTHON

```python
def process(state: SimpleState) -> dict:
    return {"outptu": state["input"].upper()}   # typo: not in the schema

app.invoke({"input": "hi", "output": "", "step": 0})
# {'input': 'hi', 'output': '', 'step': 0}   <- update dropped, no error

graph.add_edge(START, "proces")   # wrong node name
graph.compile()
# ValueError: Found edge ending at unknown node `proces`

# Checked on langgraph 1.2.6
```